



FSBM PLATFORM

Guide Technique & d'Installation

Manuel pratique pas-à-pas pour Windows

Ce guide couvre l'installation, la configuration, le lancement et l'utilisation de la plateforme FSBM en environnement Windows.

Équipe PFE — 2025/2026

AKRAM BELMOUSSA · ZAKARIA · NOUHAILA

Sommaire

Chapitre 1 — Pré-requis Windows	3
1.1 Logiciels nécessaires	3
1.2 Vérifications préalables	5
Chapitre 2 — Installation pas-à-pas	6
2.1 Téléchargement du projet	6
2.2 Méthode 1 : Setup ONE-CLICK (recommandé)	7
2.3 Méthode 2 : Installation manuelle	8
Chapitre 3 — Configuration	11
3.1 Fichiers .env	11
3.2 Base MySQL	12
3.3 MongoDB (optionnel)	13
Chapitre 4 — Lancement des services	14
4.1 Démarrage automatique	14
4.2 Démarrage manuel	15
4.3 Vérification du bon fonctionnement	16
Chapitre 5 — Référence des APIs	17
5.1 chatbot-service (port 5001)	17
5.2 academic-service (port 5002)	20
Chapitre 6 — Utilisation du frontend	23
Chapitre 7 — Dépannage (FAQ)	25
Chapitre 8 — Architecture du code	28
Annexes	30

Chapitre 1 — Pré-requis Windows

1.1 Logiciels nécessaires

Avant de commencer, vérifie que les logiciels suivants sont installés sur ton poste Windows. Si l'un manque, suis les liens officiels pour le télécharger.

Logiciel	Rôle	Source	Conseil
Python 3.10+	Recommandé : 3.12 ou 3.13. Évite 3.14 (récent, parfois incompatible).	python.org/downloads	■ Cocher « Add Python to PATH » à l'installation
Node.js 18+	Inclut npm, nécessaire pour Angular.	nodejs.org	Version LTS recommandée
MySQL 8.0+	Serveur de base de données relationnelle.	dev.mysql.com/downloads/installer	Installer aussi « MySQL Workbench »
MongoDB 7+	Optionnel pour la Phase 1 (reviews/feedbacks).	mongodb.com/try/download/community	Choisir « MongoDB Server »
Git (optionnel)	Pour cloner le projet si distribué via Git.	git-scm.com	Inclus dans Visual Studio Code
Éditeur de code	VS Code recommandé pour explorer le code source.	code.visualstudio.com	Extensions : Python, Angular Snippets

1.2 Vérifications préalables

Ouvre un terminal **PowerShell** ou **cmd.exe** et tape les commandes suivantes pour vérifier que tout est installé :

```
py --version
# Doit afficher : Python 3.12.x (ou similaire)

node --version
# Doit afficher : v18.x.x ou v20.x.x

npm --version
# Doit afficher : 9.x ou 10.x
```

■ ■ ■ **MySQL non détecté ?** C'est normal : le client mysql.exe n'est souvent pas dans le PATH. Le script SETUP.bat le détectera automatiquement.

Chapitre 2 — Installation pas-à-pas

2.1 Téléchargement du projet

Le projet est livré dans un dossier **chatbot-fsbm-platform/**. Place-le quelque part sur ton disque, par exemple :

```
C:\Users\\studies\PFE\chatbot-fsbm-platform\
```

Structure attendue :

```
chatbot-fsbm-platform/  
  ■■■ SETUP.bat           <-- Script one-click  
  ■■■ README.md  
  ■■■ docs/               Documentation + PDFs  
  ■■■ database/           Scripts SQL et générateur  
  ■■■ services/           Micro-services FastAPI  
  ■■■ frontend/           Application Angular 17  
  ■■■ scripts/            Scripts utilitaires
```

2.2 Méthode 1 — Setup ONE-CLICK (recommandé)

Le script **SETUP.bat** à la racine du projet fait **TOUT** automatiquement : config .env, install Python, install npm, init MySQL, lance les services.

■ Le SEUL input demandé est ton mot de passe MySQL. Aucune autre saisie nécessaire.

Étape 1 : Exécuter le script

Double-clique sur **SETUP.bat**, ou depuis un terminal :

```
cd C:\Users\belmo\studies\PFE\chatbot-fsbm-platform  
SETUP.bat
```

Étape 2 : Entrer ton mot de passe MySQL

Quand le script te le demande, tape ton mot de passe (utilisateur root) et appuie sur Entrée. Le mot de passe n'est pas affiché à l'écran — c'est normal.

Étape 3 : Attendre la fin (3 à 5 minutes)

Le script enchaîne automatiquement :

- [1/5] Configuration des fichiers .env (instantané)
- [2/5] Installation Python (1 minute)
- [3/5] Installation npm (1-2 minutes)
- [4/5] Chargement MySQL (30 secondes)
- [5/5] Lancement des 3 services (5 secondes)

Étape 4 : Ouvrir le navigateur

Au bout de 30 secondes, ouvre **http://localhost:4200** dans ton navigateur. Tu devrais voir le dashboard de la plateforme FSBM.

■ ■ Si une erreur survient, lis le message dans la console. Les causes les plus fréquentes sont : mot de passe MySQL incorrect, MySQL non démarré, ports 5001/5002/4200 déjà occupés.

2.3 Méthode 2 — Installation manuelle

Si tu préfères contrôler chaque étape, voici la procédure manuelle :

Étape A : Installer les dépendances Python

```
cd C:\Users\belmo\studies\PFE\chatbot-fsbm-platform\services\academic-service
py -m pip install -r requirements.txt

cd ../chatbot-service
py -m pip install -r requirements.txt
```

Étape B : Installer les dépendances Angular

```
cd ../../frontend
npm install
```

Note : npm install peut prendre 1-2 minutes selon ta connexion.

Étape C : Créer les fichiers .env

Copie **.env.example** vers **.env** dans chaque service et édite la ligne **DB_PASSWORD=** :

```
cd ../services/academic-service
copy .env.example .env
notepad .env
# Edite la ligne : DB_PASSWORD=TON_MOT_DE_PASSE

cd ../chatbot-service
copy .env.example .env
notepad .env
```

Étape D : Initialiser MySQL

Ouvre **MySQL Workbench** (recommandé) ou un terminal mysql, puis exécute dans l'ordre :

```
SOURCE database\mysql\01_schema.sql;
SOURCE database\mysql\02_seed_static.sql;
SOURCE database\mysql\03_seed_modules.sql;
SOURCE database\mysql\04_seed_data.sql;
```

Étape E : Lancer les services (3 terminaux)

Ouvre 3 fenêtres de terminal et lance dans chacune :

Terminal 1 — academic-service :

```
set PYTHONIOENCODING=utf-8
cd services\academic-service
py -m uvicorn app.main:app --reload --port 5002
```

Terminal 2 — chatbot-service :

```
set PYTHONIOENCODING=utf-8
cd services\chatbot-service
py -m uvicorn app.main:app --reload --port 5001
```

Terminal 3 — frontend Angular :

```
cd frontend
npm start
```

Chapitre 3 — Configuration

3.1 Fichiers .env

Chaque service utilise un fichier **.env** pour ses paramètres sensibles (mot de passe BDD, ports, URLs). Ces fichiers ne doivent jamais être commités.

services/academic-service/.env

```
SERVICE_NAME=academic-service
SERVICE_PORT=5002
ENV=development
DEBUG=true

DB_HOST=localhost
DB_PORT=3306
DB_NAME=fsbm_db
DB_USER=root
DB_PASSWORD=TON_MOT_DE_PASSE

CORS_ORIGINS=http://localhost:4200,http://localhost:5001
```

services/chatbot-service/.env

```
SERVICE_NAME=chatbot-service
SERVICE_PORT=5001
ENV=development
DEBUG=true

DB_HOST=localhost
DB_PORT=3306
DB_NAME=fsbm_db
DB_USER=root
DB_PASSWORD=TON_MOT_DE_PASSE

ACADEMIC_SERVICE_URL=http://localhost:5002
CONFIDENCE_THRESHOLD=0.15
MAX_HISTORY_MESSAGES=20
MAX_MESSAGE_LENGTH=500

CORS_ORIGINS=http://localhost:4200,http://localhost:3000
```

3.2 Base MySQL

Les 4 scripts SQL doivent être exécutés **dans l'ordre**. Chacun peut être chargé via MySQL Workbench (recommandé) ou en ligne de commande.

Fichier	Contenu
01_schema.sql	Crée la base fsbm_db + 16 tables avec contraintes.
02_seed_static.sql	Insère 5 départements, 7 licences, 18 masters, 16 catégories FAQ, 8 clubs, 5 événements.

03_seed_modules.sql	Insère 100+ modules détaillés par filière.
04_seed_data.sql	Génère 107 professeurs + 2970 étudiants + EDT + examens + notes (720 KB).

■ Si tu utilises MySQL Workbench, désactive le « Safe Update Mode » dans **Edit** → **Preferences** → **SQL Editor** avant de charger 04_seed_data.sql.

3.3 MongoDB (optionnel pour Phase 1)

MongoDB est utilisé en Phase 2 pour les reviews et l'analyse de sentiment. Pour l'initialiser :

```
# Depuis un terminal
mongosh < C:\Users\belmo\studies\PFE\chatbot-fsbm-platform\database\mongodb\init.js
```

Cela crée la base **fsbm_reviews** avec 6 collections + des données seed.

Chapitre 4 — Lancement des services

4.1 Démarrage automatique (recommandé)

Le script **scripts/start-all.bat** ouvre 3 fenêtres et lance les 3 services :

```
cd C:\Users\belmo\studies\PFE\chatbot-fsbm-platform
scripts\start-all.bat
```

4.2 Démarrage manuel

Voir Chapitre 2, Méthode 2, Étape E.

4.3 Vérification du bon fonctionnement

Une fois les services lancés, vérifie chaque URL dans ton navigateur :

URL	Rôle	Attendu
http://localhost:5001/api/health	Status du chatbot-service	{ status: 'ok', nlp_ready: true }
http://localhost:5001/docs	Documentation Swagger chatbot	Interface Swagger UI
http://localhost:5002/api/health	Status de l'academic-service	{ status: 'ok' }
http://localhost:5002/api/overview	Compteurs globaux	JSON avec stats des tables
http://localhost:5002/docs	Documentation Swagger academic	Interface Swagger UI
http://localhost:4200	Frontend Angular	Dashboard FSBM

Chapitre 5 — Référence des APIs

5.1 chatbot-service (port 5001)

POST /api/chat

Envoie un message au chatbot et reçoit la réponse.

Body JSON :

```
{
  "message": "Comment m'inscrire en master IADS ?",
  "session_id": "sess_abc123",    // optionnel
  "user_id": null                 // optionnel
}
```

Réponse :

```
{
  "response": "Master IADS - Intelligence Artificielle...",
  "intent": "master_iads",
  "confidence": 0.78,
  "conversation_id": 1,
  "session_id": "sess_abc123",
  "top_candidates": [...],
  "suggestions": ["Quels masters", "Master sécurité"],
  "response_time_ms": 45
}
```

POST /api/chat/feedback

Note une réponse du chatbot.

```
{
  "conversation_id": 1,
  "note": 5,                // 1 à 5
  "is_helpful": true,       // optionnel
  "commentaire": "Très clair !" // optionnel
}
```

GET /api/chat/history/{session_id}

Récupère l'historique d'une conversation.

GET /api/chat/suggestions

Retourne 6 questions suggérées pour démarrer.

GET /api/intents

Liste les 60+ intents reconnus par le chatbot.

5.2 academic-service (port 5002)

M	Endpoint	Description
---	----------	-------------

GET	/api/overview	Compteurs globaux (dashboard).
GET	/api/departments	Liste des 5 départements.
GET	/api/departments/{id}	Détail d'un département + comptes.
GET	/api/departments/{id}/filieres	Filières du département.
GET	/api/departments/{id}/professors	Profs du département.
GET	/api/filieres	Filières. Params : type, department_id, search, is_active.
GET	/api/filieres/{id}	Détail filière par ID.
GET	/api/filieres/code/{code}	Filière par code (SMI, DI, IADS...).
GET	/api/filieres/{id}/modules	Modules groupés par semestre.
GET	/api/modules	Modules. Params : filiere_id, semester, search, limit.
GET	/api/modules/{id}	Détail module avec enseignants.
GET	/api/modules/code/{code}	Module par code.
GET	/api/professors	Profs paginés. Params : department_id, grade, search, page, page_size.
GET	/api/professors/{id}	Détail professeur.
GET	/api/students	Étudiants paginés. Filtres : filiere_id, annee_etude, statut, is_boursier, search.
GET	/api/students/{id}	Détail étudiant.
GET	/api/students/cne/{cne}	Étudiant par CNE.
GET	/api/students/stats/by-filiere	Stats d'effectifs par filière.
GET	/api/schedule	EDT. Params : filiere_id, semester, annee_etude, group_name.
GET	/api/exams	Examens. Params : filiere_id, session.
GET	/api/announcements	Annonces officielles.
GET	/api/events	Événements à venir.
GET	/api/clubs	Clubs étudiants.

■ ■ Toutes les routes sont documentées et testables interactivement sur <http://localhost:5002/docs> (Swagger UI).

Chapitre 6 — Utilisation du frontend

Le frontend est accessible sur **http://localhost:4200**. Voici un tour d'horizon des 8 pages :

Page	URL	Contenu
■ Accueil (Dashboard)	/	Hero + 8 cartes stats live + actions rapides + 4 dernières annonces + 3 prochains événements.
■ Assistant IA	/chat	Interface de chat avec messages animés, suggestions rapides, feedback ■/■.
■ Départements	/departements	5 cartes des départements (Math-Info, Physique, Chimie, Biologie, Géologie).
■ Filières	/filieres	25 cartes (7 licences + 18 masters) avec recherche, filtres par type.
■ Détail filière	/filieres/:code	Programme complet par semestre + débouchés + coordinateur.
■ Modules	/modules	100+ modules avec filtres filière/semestre, recherche.
■■■ Professeurs	/professeurs	Annuaire de 107 profs paginé, recherche par nom/spécialité.
■ Actualités	/actualites	Annonces officielles + événements à venir.
■ Vie étudiante	/vie-etudiante	8 clubs étudiants avec catégories, contacts, réseaux sociaux.

Fonctionnalités transversales

- **Mode sombre** — Toggle dans la sidebar (icône lune/soleil). Persistant entre sessions (localStorage).
- **Responsive** — Sidebar passe en mode mini sur écrans < 1024px, layout adapté mobile < 768px.
- **Lazy loading** — Chaque page se charge à la demande (< 30 KB par chunk).
- **Recherche** — Disponible sur les pages Filières, Modules, Professeurs avec debounce.
- **Animations** — fadeInUp à l'affichage des cartes, hover effects, transitions de thème.
- **Live data** — Toutes les pages consomment les vraies données depuis l'academic-service.

Chapitre 7 — Dépannage (FAQ)

Q : « `mysql` : terme non reconnu » dans PowerShell

R : Le client `mysql.exe` n'est pas dans le PATH. Soit utilise le chemin complet : & "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe", soit ajoute ce dossier au PATH système via Variables d'environnement Windows.

Q : « No module named `uvicorn` » au lancement

R : Les dépendances Python ne sont pas installées. Lance : `py -m pip install -r requirements.txt` dans chaque service.

Q : « Access denied for user 'root' »

R : Le mot de passe MySQL dans `.env` est incorrect. Vérifie `services/academic-service/.env` et `services/chatbot-service/.env`. Le mot de passe doit correspondre à celui de `root@localhost`.

Q : « Duplicate entry » lors du chargement SQL

R : Tu as déjà chargé les données. Lance d'abord : `DROP DATABASE fsbm_db`; puis re-exécute les 4 scripts dans l'ordre. Le script `01_schema.sql` commence par `DROP DATABASE IF EXISTS`.

Q : « Safe update mode » dans Workbench

R : Workbench bloque les UPDATE sans WHERE. Va dans Edit > Preferences > SQL Editor et décoche « Safe Updates ». Puis Query > Reconnect to Server.

Q : « UnicodeEncodeError » dans la console Python

R : Le terminal Windows utilise `cp1252` par défaut. Lance d'abord : `set PYTHONIOENCODING=utf-8` avant la commande Python.

Q : Le port 5001/5002/4200 est déjà utilisé

R : Un autre programme occupe le port. Trouve-le avec : `netstat -ano | findstr :5001` puis tue le processus avec son PID. Ou change le port dans le `.env`.

Q : Le chatbot répond toujours « Je ne comprends pas »

R : Le seuil de confiance peut être trop élevé. Vérifie `CONFIDENCE_THRESHOLD=0.15` dans `chatbot-service/.env`. Sinon, vérifie que `faq_dataset.json` existe bien dans `services/chatbot-service/data/`.

Q : Le dashboard affiche un warning « backend non joignable »

R : L'`academic-service` ne tourne pas ou est sur un autre port. Vérifie qu'il répond sur `http://localhost:5002/api/health` avant de recharger le frontend.

Q : Les emojis ne s'affichent pas dans le terminal

R : C'est uniquement un problème d'affichage du terminal cmd.exe. Les emojis sont bien dans la base et s'affichent correctement dans le navigateur.

Q : npm install est très lent

R : Vérifie ta connexion. Tu peux essayer un autre registry : `npm config set registry https://registry.npmirror.com/`.

Q : « Cannot find module './app/main' »

R : Tu lances uvicorn depuis le mauvais dossier. Vérifie que tu es bien dans `services/academic-service/` ou `services/chatbot-service/` avant de lancer.

Chapitre 8 — Architecture du code source

8.1 Structure d'un service FastAPI

```
services/academic-service/
■■■■ app/
■   ■■■■ main.py           <-- Point d'entree, lifespan, CORS
■   ■■■■ core/
■   ■   ■■■■ config.py     <-- Pydantic Settings depuis .env
■   ■■■■ db/
■   ■   ■■■■ session.py    <-- Engine async, sessionmaker
■   ■■■■ models/
■   ■   ■■■■ entities.py   <-- ORM SQLAlchemy 2.0
■   ■■■■ schemas/
■   ■   ■■■■ academic.py   <-- Validation Pydantic v2
■   ■■■■ routers/         <-- 8 modules de routes
■■■■ .env                 <-- Configuration secrete
■■■■ .env.example         <-- Template
■■■■ requirements.txt
```

8.2 Structure du frontend Angular

```
frontend/src/app/
■■■■ app.component.ts      <-- Racine de l'application
■■■■ app.routes.ts        <-- Definition des routes (lazy)
■■■■ layout/
■   ■■■■ app-shell.component.ts <-- Sidebar + topbar + outlet
■   ■■■■ app-shell.component.css
■■■■ core/
■   ■■■■ theme.service.ts   <-- Mode sombre/clair
■■■■ services/
■   ■■■■ chat.service.ts    <-- Client HTTP chatbot-service
■   ■■■■ academic.service.ts <-- Client HTTP academic-service
■■■■ features/             <-- Pages lazy-loaded
■   ■■■■ dashboard/
■   ■■■■ chat/
■   ■■■■ departments/
■   ■■■■ filieres/
■   ■■■■ modules/
■   ■■■■ professors/
■   ■■■■ news/
■   ■■■■ student-life/
■■■■ components/           <-- Atomiques reutilisables
■■■■ models/
```

8.3 Bonnes pratiques de développement

- Toujours créer un nouveau `.env` à partir de `.env.example`, jamais commiter `.env`.
- Utiliser Pydantic v2 pour valider tous les inputs (déjà fait dans le code livré).
- Pour ajouter un nouveau intent au chatbot : éditer `services/chatbot-service/data/faq_dataset.json`, puis supprimer `model.pkl` pour forcer le réentraînement.

- Pour ajouter une nouvelle route Angular : créer le composant standalone dans features/, ajouter une entrée dans app.routes.ts avec loadComponent.
- Pour ajouter une table MySQL : modifier 01_schema.sql + ajouter le modèle ORM dans models/entities.py + le schéma Pydantic + le router.
- Toujours tester localement avant de pousser du code.

Annexes

Annexe A — Liste des fichiers livrés

chatbot-fsbm-platform/	(racine)
■■■■ SETUP.bat	Setup one-click
■■■■ README.md	README principal
■■■■ docs/	
■ ■■■■ ARCHITECTURE.md	Architecture détaillée
■ ■■■■ pdf/	
■ ■■■■ FSBM_Platform_Rapport_PFE.pdf	PDF 1 : Rapport
■ ■■■■ FSBM_Platform_Guide_Technique.pdf	PDF 2 : Ce guide
■ ■■■■ generate_report.py	Script generateur PDF 1
■ ■■■■ generate_guide.py	Script generateur PDF 2
■■■■ database/	
■ ■■■■ mysql/	
■ ■ ■■■■ 01_schema.sql	(18 KB)
■ ■ ■■■■ 02_seed_static.sql	(19 KB)
■ ■ ■■■■ 03_seed_modules.sql	(15 KB)
■ ■ ■■■■ 04_seed_data.sql	(~720 KB, genere)
■ ■■■■ mongodb/init.js	(13 KB)
■ ■■■■ seed/generate_data.py	(Python generator)
■■■■ services/	
■ ■■■■ chatbot-service/	(FastAPI :5001)
■ ■■■■ academic-service/	(FastAPI :5002)
■■■■ frontend/	(Angular 17)
■■■■ scripts/	
■ ■■■■ install-all.bat	
■ ■■■■ start-all.bat	
■ ■■■■ init-database.bat	
■ ■■■■ configure-env.bat	

Annexe B — Commandes utiles

Réentraîner le NLP

```
del services\chatbot-service\data\model.pkl
Redémarrer le chatbot-service
```

Régénérer les données seed

```
cd database\seed
set PYTHONIOENCODING=utf-8
py generate_data.py
```

Builder le frontend pour production

```
cd frontend
npm run build:prod
```

Vider la BDD

```
DROP DATABASE fsbm_db;
```

Vérifier les tables

```
SELECT TABLE_NAME, TABLE_ROWS FROM information_schema.TABLES WHERE TABLE_SCHEMA='f
sbm_db';
```

Tester le chatbot via curl

```
curl -X POST http://localhost:5001/api/chat \  
-H "Content-Type: application/json" \  
-d "{\"message\":\"Quelles sont les filieres ?\"}"
```

Annexe C — Crédits & contact

Cette plateforme a été développée dans le cadre du Projet de Fin d'Études de la Licence en Développement Informatique à la Faculté des Sciences Ben M'Sick (Université Hassan II de Casablanca), année universitaire 2025/2026.

Équipe :

- AKRAM BELMOUSSA — Architecte Backend & NLP
- ZAKARIA — Frontend Angular & UX/UI
- NOUHAILA — Base de données & Contenu FAQ

■ Pour toute question ou amélioration, contactez l'équipe ou consultez la documentation complète dans **docs/ARCHITECTURE.md**.